Lecture 07：语音合成基础（TTS Foundation）

从文字到声音：前端、声学模型、声码器与评价

王帅

南京大学智能科学与技术学院

2026 年 10 月 23 日

本节学习目标

1. 说清语音合成（TTS）的输入输出、典型应用，以及“好的合成语音”的标准
2. 理解 TTS 的四个核心难题：读什么、读多长、怎么读、怎么发声
3. 掌握经典三段式流水线：文本前端 $\rightarrow$ 声学模型 $\rightarrow$ 声码器
4. 理解 Tacotron 2、FastSpeech 2、WaveNet、HiFi-GAN、VITS 分别解决了哪个难题
5. 会用 MOS、CER/WER、SIM、RTF 评价一个 TTS 系统

本讲核心问题

同一句话可以有无数种读法。机器如何从一串文字出发，“想象”出一段读得对、听着自然的声音？

本讲结构

基础与传统方法

1. 任务定义与四个核心难题
2. 文本前端：读什么
3. 传统合成：拼接与统计参数

神经网络语音合成

4. 声学模型：从文字到频谱
5. 声码器：从频谱到波形
6. 端到端合成：VITS
7. 评价方法与展望

贯穿全讲的主线

每个模型都在回答四个核心难题中的一个或几个。遇到新模型时，先问：它解决了哪个难题？代价是什么？

目录

1. 任务定义与核心难题
2. 文本前端：读什么
3. 传统语音合成：拼接与统计参数
4. 声学模型：从文字到频谱
5. 声码器：从频谱到波形
6. 端到端合成：VITS
7. 如何评价一个 TTS 系统
8. 展望：大模型时代的 TTS
9. 总结

ASR 与 TTS：一对“逆问题”



	ASR	**TTS**
任务类型	识别：连续信号 → 离散符号	生成：离散符号 → 连续信号
“正确答案”	通常只有一个正确文本	有无数种正确读法
映射关系	多对一：很多声音对应同一句话	一对多：一句话对应很多声音
主要评价	WER / CER	MOS、可懂度、音色相似度、速度

什么是语音合成？

定义

语音合成（Text-to-Speech, TTS）：把任意输入文本转换成可懂、自然的语音波形。核心挑战是：只用有限的录音训练，却要能读出从没听过的句子。

典型应用

- 信息无障碍：读屏软件、视障辅助
- 导航、智能音箱、车载助手
- 有声书、短视频配音、新闻播报
- 语音对话助手（“能说话的大模型”）
- 虚拟人、游戏角色、同声传译配音

本讲的符号约定

- 文本 / 音素序列：$x_{1:N}$
- 声学特征（Mel 频谱）：$y_{1:T}$，每帧 80 维
- 波形采样点：$s_{1:L}$
- 通常 $N \ll T \ll L$

什么样的合成语音算“好”？

可懂度

读对了吗？

字词不错读、不漏读、不重复

自然度

像真人吗？

音质干净，停顿、语调、节奏自然

表现力

像“那个人”吗？

音色、情感、风格是否符合要求

效率

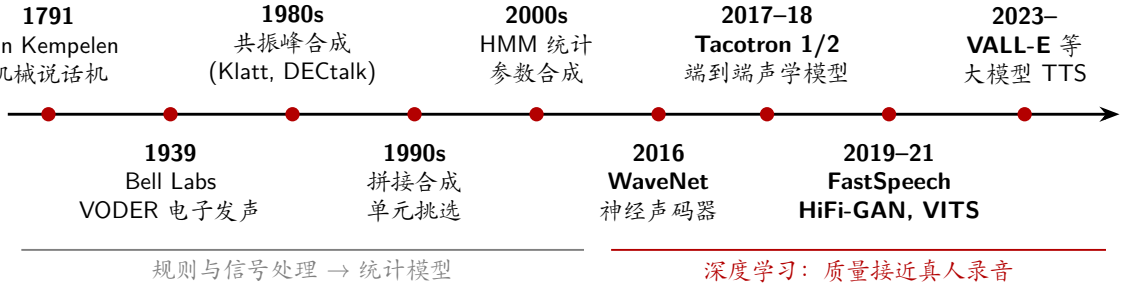
够快吗？

实时率、首包延迟、能否在手机上运行

记住这四个维度

本讲最后的“评价”一节会把它们分别对应到具体指标：**CER/WER**、**MOS**、**SIM**、**RTF**。

语音合成发展简史



一条清晰的演进逻辑

每一代方法都在弥补上一代最明显的短板：拼接合成音质好但不灵活 → 统计参数合成灵活但发闷 → 神经网络兼顾音质与灵活 → 大模型追求“任何声音、任何风格”。

从文字到声音，要回答四个问题

①读什么？(发音)

“110” 读“幺幺零”还是“一百一十”？“银行” 的 “行” 读什么？
⇒ 文本前端

②读多长？(对齐 / 时长)

6 个字要变成约 150 帧频谱，每个字分到几帧？
⇒ 注意力机制或时长模型

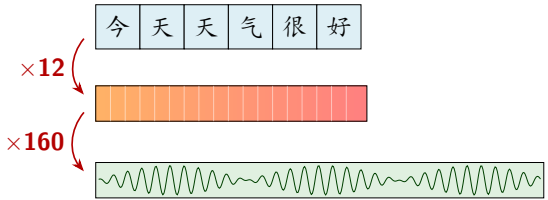
③怎么读？(韵律 / 一对多)

在哪停顿？语调升还是降？高兴还是难过？谁的声音？
⇒ 韵律建模、生成模型

④怎么发声？(波形)

每秒要生成 16000–48000 个采样点，还要补回相位信息
⇒ 声码器

数一数：文字、频谱与波形差了几个数量级



文本：6 个汉字 $\approx$ 12 个声母/韵母

Mel 频谱：1.5 秒 $\times$ 100 帧/秒 $=$ 150 帧 $\times$ 80 维

波形：1.5 秒 $\times$ 16000 点/秒 $=$ 24000 个采样点

直接生成为什么难？

从 12 个符号直接“想象”出 24000 个数：序列极长、结构复杂、误差容易累积。

经典思路：分而治之

先生成一个紧凑的中间表示（Mel 频谱），再把它“翻译”成波形。

经典三段式流水线



图源：X. Tan et al., A Survey on Neural Speech Synthesis, 2021

模块	输入 → 输出	例子	回答的问题
文本前端	文字 → 语言学特征	今天 → j in1 t ian1	①读什么
声学模型	语言学特征 → Mel 频谱	12 个音素 → 150 帧	②读多长、③怎么读
声码器	Mel 频谱 → 波形	150 帧 → 24000 点	④怎么发声

拼接合成、统计参数合成、神经网络合成都可以放进这个框架里理解，区别在于每个模块用什么方法实现。

中间表示为什么选 Mel 频谱？

【回顾】第 03 讲：分帧、加窗、STFT、Mel 滤波器组、取对数 ⇒ log-Mel 频谱

Mel 频谱的优点

- 紧凑：每秒 100 帧 × 80 维，远小于每秒 16000 个采样点
- 贴近听感：Mel 刻度模拟人耳的频率分辨率
- 平滑、易预测：相邻帧变化缓慢，适合神经网络回归

但它丢掉了信息

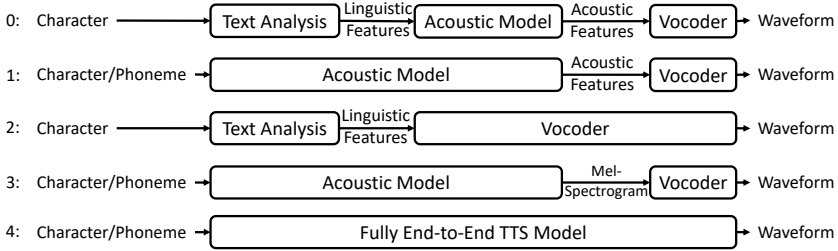
- 只保留幅度，丢掉了相位
- 80 个 Mel 频带合并了大量线性频率格点

⇒ 需要声码器把缺失的信息“补”回来

一个类比

Mel 频谱像乐谱：声学模型负责“作曲”（决定什么时候、什么音高、多响），声码器负责“演奏”（把乐谱变成真实的声音）。

从三段式到全端到端



图源：X. Tan et al., A Survey on Neural Speech Synthesis, 2021

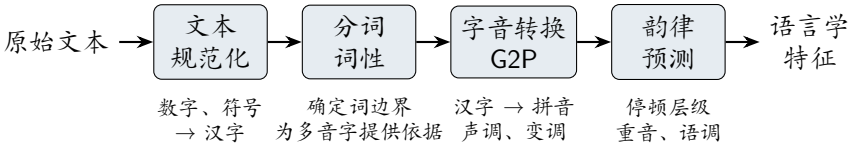
演进趋势

- 模块越来越少：第 3 行（声学模型 + 声码器）是神经 TTS 最常见的形态，第 4 行（如 VITS）直接从文本生成波形
- 但无论模块怎么合并，四个核心难题始终存在，只是由不同部件“隐式”承担

目录

1. 任务定义与核心难题
2. 文本前端：读什么
3. 传统语音合成：拼接与统计参数
4. 声学模型：从文字到频谱
5. 声码器：从频谱到波形
6. 端到端合成：VITS
7. 如何评价一个 TTS 系统
8. 展望：大模型时代的 TTS
9. 总结

文本前端做什么？



为什么需要前端？

- 文字和发音并不一一对应
- 前端把“写法”翻译成“读法”，为声学模型提供明确的发音信息

错误会被直接“读出来”

ASR 的错误要看文字才能发现；TTS 前端读错一个多音字，听众立刻就能听出来。

文本规范化（Text Normalization, TN）

任务

把数字、符号、缩写等非标准词转换成可以直接朗读的文字。

原文	规范化结果	依据的上下文
2026 年 10 月 23 日	二零二六年十月二十三日	年份逐位读，日期按数值读
拨打 **110**	拨打幺幺零	电话号码
110 米栏	一百一十米栏	数量
比分 **2:1**	比分二比一	比分
8:30 上课	八点三十上课	时间
增长 3.5%	增长百分之三点五	百分数
Dr. Smith / Baker St.	Doctor Smith / Baker Street	英文缩写

关键观察

同一个符号的读法取决于上下文。工业系统通常用规则或加权有限状态转换器（WFST，如 WeTextProcessing）处理，再用神经网络处理有歧义的情况。

字音转换（Grapheme-to-Phoneme, G2P）

中文难点一：多音字

字	读法 **A**	读法 **B**
行	银行 háng	行走 xíng
长	长大 zhǎng	长度 cháng
重	重新 chóng	重要 zhòng
乐	音乐 yuè	快乐 lè
还	还钱 huán	还是 hái

中文难点二：变调

- 三声连读：你好 nǐ hǎo → **ní** hǎo
- “一”：一天 yì、一个 yí、第一 yī
- “不”：不好 bù、不是 bú

英文难点

- 拼写与发音不规则：though / through / tough
- 同形异音：read（现在 / 过去）、record（名词 / 动词）
- 未登录词：人名、地名、新词

常用方法

- 词典查找：CMUdict、拼音词典（覆盖常见词）
- 统计 / 神经模型：预测未登录词，结合上下文为多音字消歧（如 g2pW 使用 BERT）

韵律预测：停顿、重音与语调

韵律（Prosody）

同样的字、同样的发音，停顿位置、重音、语调、语速不同，听起来的意思和自然度就完全不同。

停顿改变语义："咬死了猎人的狗"

- 咬死了 # 猎人的狗
  ⇒ 狗（属于猎人）被咬死了
- 咬死了猎人的 # 狗
  ⇒ 这条狗咬死了猎人

常见的停顿层级标注

#1　韵律词（几乎不停顿）
#2　韵律短语（短停顿）
#3　语调短语（明显停顿）
#4　句末

中文开源数据集"标贝"采用了这种标注

韵律是自然度的关键

韵律预测得不好，每个字都读对了，听起来仍然像"机器在念字"。

完整走一遍：前端的输出长什么样？

输入　2026 年 10 月 23 日，我们开始学习语音合成。

①文本规范化　二零二六年十月二十三日，我们开始学习语音合成。

②分词 + 韵律预测

二零二六年 #1 十月 #1 二十三日 #3 我们 #1 开始 #2 学习 #1 语音 #1 合成 #4

③ **G2P**：送入声学模型的音素序列（节选）

```
... w o3 m en5 #1 k ai1 sh i3 #2 x ve2 x i2 #1 y v3 y in1 #1 h e2
ch eng2 #4
```

课堂练习：做一次“人肉前端”

请写出下面句子的朗读形式，并标出多音字的读音

他在银行工作了 3.5 年，每天 7:30 出门，长得很高。

课堂练习：做一次“人肉前端”

请写出下面句子的朗读形式，并标出多音字的读音

他在银行工作了 3.5 年，每天 7:30 出门，长得很高。

参考答案

- 3.5 年 → 三点五年；7:30 → 七点三十（或“七点半”）
- 银行：háng；工作了：le；长得很高：zhǎng

课堂练习：做一次“人肉前端”

请写出下面句子的朗读形式，并标出多音字的读音

他在银行工作了 3.5 年，每天 7:30 出门，长得很高。

参考答案

- 3.5 年 → 三点五年；7:30 → 七点三十（或“七点半”）
- 银行：háng；工作了：le；长得很高：zhǎng

思考

“七点三十”和“七点半”都对。这说明连“读什么”也不总是只有一个答案。端到端模型直接输入字符时，这些判断就要由模型自己从数据中学会。

目录

1. 任务定义与核心难题
2. 文本前端：读什么
3. 传统语音合成：拼接与统计参数
4. 声学模型：从文字到频谱
5. 声码器：从频谱到波形
6. 端到端合成：VITS
7. 如何评价一个 TTS 系统
8. 展望：大模型时代的 TTS
9. 总结

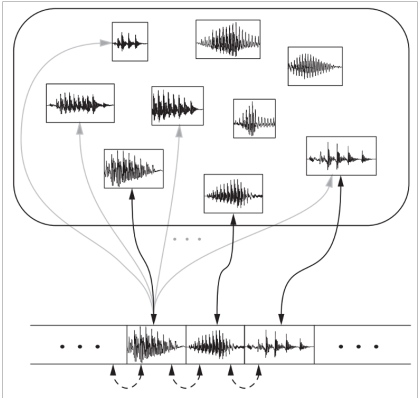
拼接合成（Concatenative Synthesis）

基本思想：“剪贴录音”

1. 请一位播音员录制大量语音（数小时到数十小时）
2. 切分成小单元（音素、半音节、音节……）存入音库
3. 合成时为每个目标单元挑选最合适的录音片段
4. 把片段拼接起来，并对接缝处做平滑处理

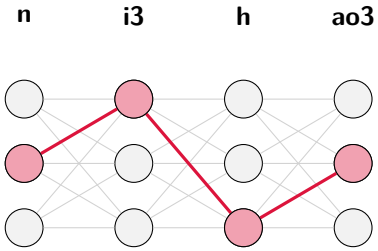
为什么音质好？

每个片段都是真实录音，音色天然逼真。



上：音库中的候选单元；下：拼接得到的句子

单元挑选：在格子图里找一条最优路径



目标：“你好”；每列 3 个候选录音片段

两种代价

$$C = \sum_i \underbrace{C^t(u_i, t_i)}_{\text{目标代价}} + \sum_i \underbrace{C^c(u_{i-1}, u_i)}_{\text{拼接代价}}$$

- 目标代价：候选片段的音高、时长、上下文是否符合要求
- 拼接代价：相邻两段接起来是否顺滑

【回顾】第 04 讲的维特比算法：用动态规划在格子图中找总代价最小的路径

生活中的“拼接合成”：鬼畜视频



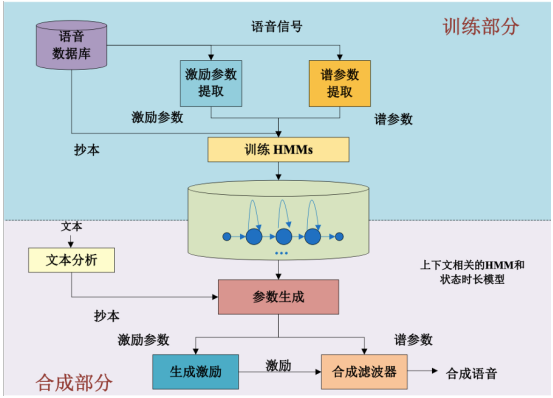
把已有录音切成字、词，再按新文本重新拼接，就是最朴素的拼接合成。接缝处的音高、音色、能量跳变正是拼接合成的典型缺陷。

统计参数语音合成（SPSS）

基本思想：不存录音，存“参数模型”

1. 分析：把录音转换成声学参数（频谱包络 + 基频 F0）
2. 建模：用上下文相关 HMM 学习“文本 → 参数”的统计规律
3. 合成：根据文本预测参数轨迹，再由声码器（源-滤波器模型）生成波形

【回顾】第 04 讲的三音素与决策树状态绑定；第 03 讲的源-滤波器模型和 $\Delta/\Delta\Delta$ 特征



SPSS 为什么听起来“发闷、有机器味”？

原因一：过平滑（Over-smoothing）

- HMM 输出的是统计意义上的平均参数
- 平均会抹掉真实语音中的细节和变化
- 频谱变模糊 ⇒ 声音发闷；音高变平 ⇒ 语调单调

原因二：声码器过于简化

- 源-滤波器模型只是对发声过程的近似
- 激励用“脉冲 + 噪声”表示，相位信息粗糙
- 声音带有“嗡嗡”的蜂鸣感（buzzy）

请记住“过平滑”这个词

它是难题③（一对多）的第一次出场：一句话有很多种读法，模型却只学到了“平均读法”。后面的 FastSpeech 2、生成模型、VITS 都在和它作斗争。

传统方法对比

	拼接合成	统计参数合成	神经网络合成
基本原理	挑选并拼接真实录音	HMM 预测参数 + 声码器	深度网络学习映射
音质	高（接缝处有瑕疵）	低（发闷、机器味）	接近真人
灵活性 / 可控性	差：改风格需重录	好：改参数即可	好
存储	大（整个音库）	小（模型参数）	中
典型问题	接缝不连续	过平滑	鲁棒性、速度

神经网络 TTS 要达成的目标

拼接合成的音质 + 统计参数合成的灵活性

目录

1. 任务定义与核心难题
2. 文本前端：读什么
3. 传统语音合成：拼接与统计参数
4. 声学模型：从文字到频谱
5. 声码器：从频谱到波形
6. 端到端合成：VITS
7. 如何评价一个 TTS 系统
8. 展望：大模型时代的 TTS
9. 总结

声学模型的任务：难题②与③

问题设定

输入音素序列 $x_{1:N}$，输出 Mel 频谱 $y_{1:T}$。其中 $T \gg N$（约 10–20 倍），而且每个音素对应哪些帧是未知的。

路线 A：自回归 + 注意力

边生成边对齐：每生成一帧，用注意力决定当前该读哪个音素。

代表：**Tacotron 2**（2018）

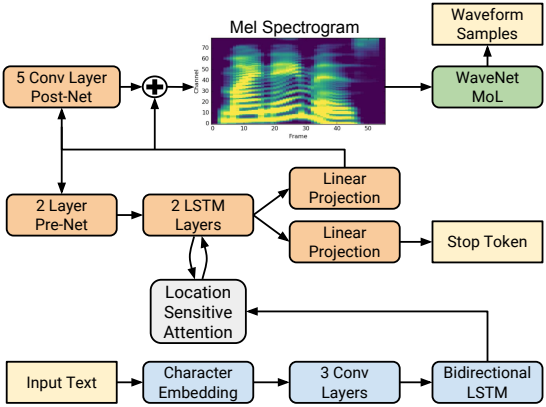
路线 B：非自回归 + 显式时长

先定时长，再并行生成：先预测每个音素读几帧，再一次性生成所有帧。

代表：**FastSpeech 1/2**（2019–2021）

【回顾】第 05 讲：ASR 里的 AED 模型也用注意力解决“语音帧与文字不等长”的问题，TTS 只是把方向反了过来。

Tacotron 2：整体架构



图源：J. Shen et al., ICASSP 2018

编码器（蓝色）：读文本

字符嵌入 → 3 层卷积 → 双向 LSTM，得到每个字符的上下文表示

注意力：决定“现在读到哪”

位置敏感注意力会参考上一步关注的位置，使对齐尽量向前推进

解码器（橙色）：逐帧写频谱

每步输出一帧 Mel 和一个停止标记（stop token）；后处理网络修正细节，最后由 WaveNet 声码器生成波形

Tacotron 2：训练与推理

自回归分解：每一帧都依赖之前所有帧和整个输入文本

$$p(y_{1:T} \mid x) = \prod_{t=1}^{T} p(y_t \mid y_{<t}, x)$$

训练：Teacher Forcing

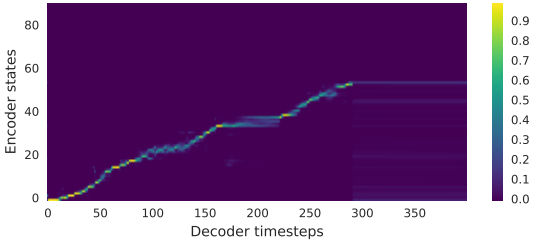
- 解码器输入真实的上一帧，可并行计算
- 损失：Mel 均方误差 + 停止标记交叉熵

推理：逐帧生成

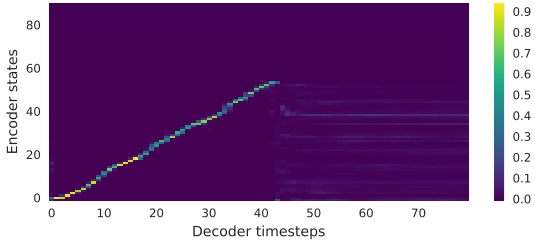
- 输入自己生成的上一帧
- 必须串行：1 秒语音约 80–100 步
- 停止标记概率 > 0.5 时停止

效果：英文单说话人上，Tacotron 2 的 MOS 为 **4.53**，真人录音为 **4.58**，神经 TTS 首次在主观评分上接近真人。

注意力对齐：一条“单调的对角线”



对齐失败：注意力分散、跳跃



对齐成功：清晰的对角线

图源：Y. Wang et al., Tacotron, Interspeech 2017

怎么读这张图？

横轴是解码步（生成的帧），纵轴是编码器位置（文本）。亮点表示“生成这一帧时在读哪个字”。人朗读时是从左到右、不回头、不跳字的，所以好的对齐应该是单调递增的对角线。

自回归 + 注意力的代价

问题 1：慢

推理必须一帧一帧串行生成，句子越长越慢，难以满足实时需求。

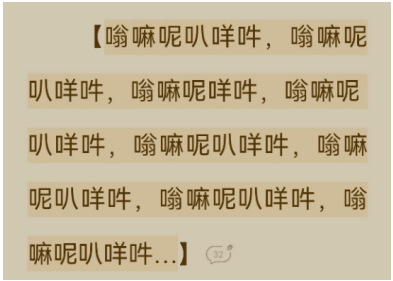
问题 2：不稳（鲁棒性差）

注意力一旦“走错”，错误会直接被听到：

- 跳词：注意力跳过了某些字
- 重复：注意力卡在某处来回打转
- 停不下来：停止标记一直没有触发

问题 3：难控制

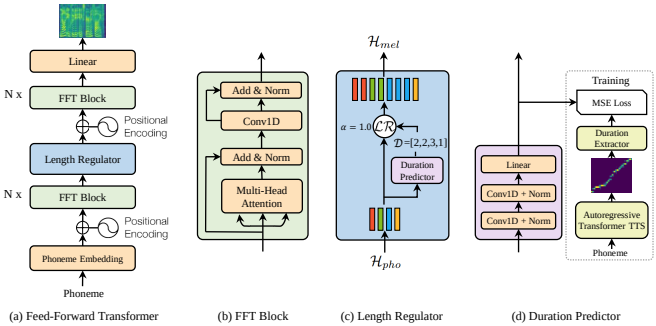
时长是注意力“隐式”决定的，想让某个字读慢一点很难做到。



思考

这样大量重复的生僻字文本，对“边读边找位置”的注意力模型意味着什么？

FastSpeech：先定时长，再并行生成



(a) Feed-Forward Transformer (b) FFT Block (c) Length Regulator (d) Duration Predictor

图源：Y. Ren et al., FastSpeech, NeurIPS 2019

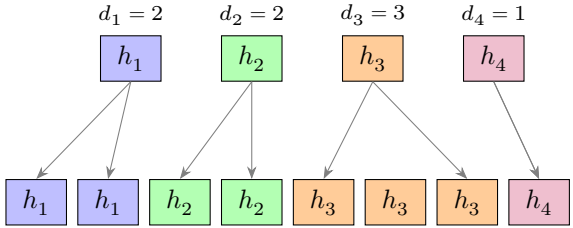
核心组件

- **FFT Block**：自注意力 + 1D 卷积（图 b）
- 长度调节器：按时长把音素表示“复制展开”（图 c）
- 时长预测器：预测每个音素读几帧（图 d）

关键变化

对齐不再由注意力“边走边找”，而是显式地由时长决定。帧与帧之间不再相互依赖，可以一次性并行生成。

长度调节器（Length Regulator）：手算一遍



4 个音素的表示

8 帧，送入 Mel 解码器

规则

时长 $\mathbf{d} = [2, 2, 3, 1]$：每个 h_i 复制 d_i 次，输出长度 $T = \sum_i d_i = 8$。

语速控制：$d'_i = \mathrm{round}(\alpha\, d_i)$

$\alpha = 1.5$ 时 $\mathbf{d}' = [3, 3, 5, 2]$（读得更慢）；$\alpha < 1$ 时读得更快。不需要重新训练模型。

训练时，每个音素的时长从哪里来？

FastSpeech：向“老师”要

- 先训练一个自回归 Transformer TTS（教师模型）
- 从它最接近对角线的注意力图中读出时长
- 步骤繁琐，时长精度受教师模型限制

FastSpeech 2：用强制对齐

- 用 Montreal Forced Aligner（MFA）等工具
- 已知文本和录音，求每个音素的起止时间

【回顾】这正是第 04 讲 HMM 维特比强制对齐的用途

时长预测器

- 结构很简单：2 层 1D 卷积 + 线性层
- 训练目标：预测对数时长，用均方误差

$$\mathcal{L}_{\text{dur}} = \left(\log \hat{d}_i - \log d_i\right)^2$$

- 推理：用预测的 $\hat{d}_i$ 取整后送入长度调节器

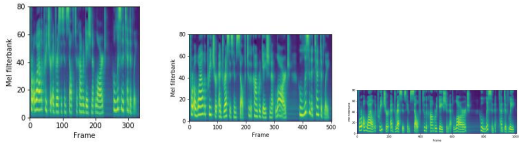
为什么用对数？

时长分布偏斜，长音素差几帧和短音素差几帧，听感影响不同。

FastSpeech 的收获：快、稳、可控

论文报告的结果（英文 LJSpeech）

- Mel 频谱生成速度：比自回归基线快约 **270** 倍
- 端到端（含声码器）速度：快约 **38** 倍
- 50 句困难句子的错误率：Transformer TTS 为 34%，FastSpeech 为 **0%**
- 音质（MOS）与自回归模型接近



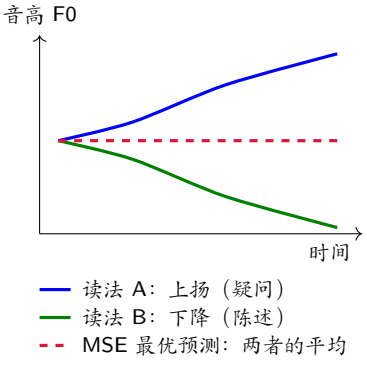
$\alpha = 0.5$（快）　　$\alpha = 1.0$　　$\alpha = 1.5$（慢）

同一句话在不同语速下的 Mel 频谱

为什么不再跳词、重复？

每个音素都被分配了 $d_i \geq 1$ 帧，结构上保证了每个音素恰好读一次、按顺序读。

难题③：一对多，以及“平均”的陷阱



同一句“你明天去”，两种读法都正确

一个最简单的计算

若目标 $y = +1$ 和 $y = -1$ 各占一半，最小化均方误差 $\mathbb{E}[(y - \hat{y})^2]$ 的预测是

$$\hat{y} = \mathbb{E}[y] = 0$$

这是一个两种读法都不是的答案。

后果

用 L1/L2 损失回归 Mel 频谱，会得到“平均读法”：语调平、频谱糊，即前面说的过平滑。

对付一对多：两种思路

思路 1：补充信息

告诉模型更多条件，让“答案”变得更确定：

- 音高、能量曲线 ⇒ **FastSpeech 2**
- 说话人身份 ⇒ 多说话人 **TTS**
- 参考音频的风格 ⇒ **GST**

条件越多，剩下的不确定性越小，“平均”带来的模糊就越轻。

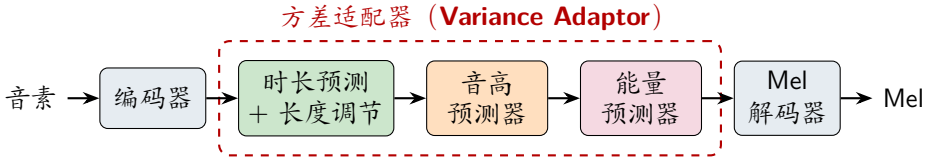
思路 2：学习分布

不再预测一个平均值，而是学习整个分布 $p(y \mid x)$，再从中采样：

- 自回归、GAN、流模型、扩散模型
- 声码器一节会看到它们的直观思想
- VITS 和大模型 TTS 都属于这一思路

采样得到的是某一种真实可能的读法，而不是“四不像”。

FastSpeech 2：用方差适配器补充信息



训练与推理

- 训练：直接用从录音中提取的真实时长、音高、能量作为条件，同时训练各个预测器
- 推理：用预测值代替真实值
- 直接用真实 Mel 训练，去掉了教师模型

好处

- 音高、能量给定后，“怎么读”的不确定性变小，过平滑减轻
- 可控：手动调高音高曲线、放大能量即可改变语气
- 训练比 FastSpeech 快约 3 倍

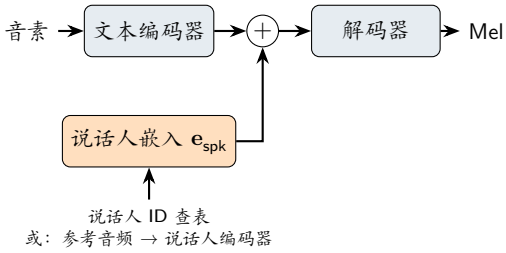
两条路线的对比

	自回归 + 注意力（**Tacotron 2**）	非自回归 + 时长（**FastSpeech 2**）
生成方式	逐帧串行	所有帧并行
对齐方式	注意力隐式学习	显式时长（外部对齐器提供）
推理速度	慢	快
鲁棒性	可能跳词、重复	结构上避免
可控性	弱	强（时长、音高、能量）
韵律多样性	自回归建模，较自然	依赖预测器，偏“平均”
额外依赖	无	需要强制对齐工具

没有免费的午餐

路线 B 用“额外的对齐工具和人为设计的特征”换来了速度和稳定性。后面的 VITS 会尝试在模型内部自动学到对齐。

多说话人：让模型换一种声音



两种获得音色向量的方式

- 查表：训练集中每个说话人学一个向量，只能合成见过的人
- 说话人编码器：从一段参考音频中提取向量，可以模仿没见过的人（零样本声音克隆）

【回顾】第 06 讲的 d-vector / x-vector：说话人识别模型学到的嵌入可以直接迁移给 TTS（Jia et al., 2018）

同样的思路可以扩展到风格与情感

例如 Global Style Tokens（GST）：从参考音频中提取“风格向量”，推理时可以用参考音频，也可以手动组合风格 token。

目录

1. 任务定义与核心难题
2. 文本前端：读什么
3. 传统语音合成：拼接与统计参数
4. 声学模型：从文字到频谱
5. 声码器：从频谱到波形
6. 端到端合成：VITS
7. 如何评价一个 TTS 系统
8. 展望：大模型时代的 TTS
9. 总结

声码器的任务：难题④

任务

输入 Mel 频谱（如每秒 86 帧 × 80 维），输出波形（如每秒 22050 个采样点），每帧要生成 **256** 个采样点。

难点 1：信息不够

- Mel 频谱没有相位
- 80 个频带比原始频谱粗糙得多
- 同一张 Mel 频谱对应许多可能的波形

⇒ 又是一个一对多问题

难点 2：序列太长

- 1 秒就有上万个采样点
- 人耳对波形细节非常敏感，很小的误差就会听成杂音
- 实际应用还要求实时

演进主线：信号处理 → 自回归神经网络（质量突破）→ 并行神经网络（又好又快）

信号处理声码器：Griffin-Lim 与 WORLD

Griffin-Lim（1984）：迭代估计相位

1. 已知幅度谱，先随机初始化相位
2. 逆 STFT 得到波形，再做 STFT
3. 保留新相位，把幅度换回已知值
4. 重复步骤 2–3 直到收敛

特点：无需训练；音质有“金属感”。Tacotron 1 使用的就是它。

WORLD 等参数声码器

【回顾】第 03 讲的源-滤波器模型

把语音分解为三组参数：

- 基频 F0（声源）
- 频谱包络（声道滤波器）
- 非周期成分（噪声）

特点：快、可解释；但模型过于简化，有“嗡嗡声”。

共同瓶颈

依赖人为设计的假设，音质有天花板。神经声码器的思路是：不做假设，让网络直接从数据中学习波形。

生成模型小地图（只需记住直觉）

类别	一句话直觉	**TTS** 中的代表
自回归模型	一个接一个地生成，每一步都看前面的结果	WaveNet、Tacotron 2、VALL-E
流模型（Flow）	用可逆变换把简单噪声“揉”成复杂数据	WaveGlow、Glow-TTS
对抗网络（GAN）	生成器造假，判别器打假，二者共同进步	MelGAN、HiFi-GAN
变分自编码器（VAE）	把数据压缩成隐变量，再从隐变量重建	VITS（与 Flow、GAN 结合）
扩散 / 流匹配	从纯噪声出发，一步步去噪得到数据	DiffWave、Grad-TTS、F5-TTS

它们的共同目标

不再预测一个“平均答案”，而是学习数据的整个分布 $p(\text{语音} \mid \text{条件})$，然后从中采样。这正是解决一对多问题的第二种思路。

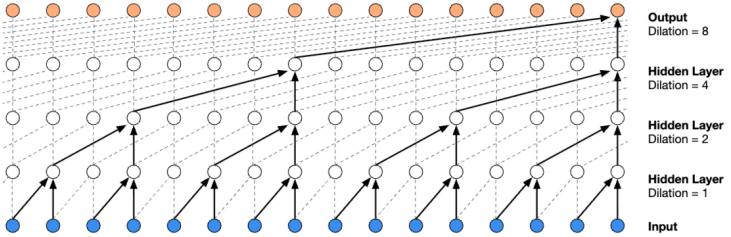
数学推导（ELBO、雅可比行列式、得分函数等）会在深度学习、生成模型相关课程中系统学习。

WaveNet（2016）：逐个采样点生成

自回归建模波形

$$p(s_{1:L} \mid c) = \prod_{t=1}^{L} p(s_t \mid s_{<t}, c), \qquad c = \text{Mel 频谱}$$

采样值经 μ-law 压缩量化为 256 个等级，把“预测一个实数”变成 **256** 类分类问题。



空洞因果卷积：因果指只看过去，空洞（dilation）指跳着看，使感受野指数增长

课堂练习：WaveNet 能“看”多远？

感受野公式

卷积核大小为 k、空洞率依次为 $d_1, d_2, \ldots$ 的因果卷积堆叠，感受野为

$$R = 1 + \sum_{l} (k-1)\, d_l$$

问题：$k = 2$，空洞率为 $1, 2, 4, \ldots, 512$（共 10 层），16 kHz 采样，感受野是多少？

课堂练习：WaveNet 能“看”多远？

感受野公式

卷积核大小为 k、空洞率依次为 $d_1, d_2, \ldots$ 的因果卷积堆叠，感受野为

$$R = 1 + \sum_{l} (k-1)\, d_l$$

问题：$k = 2$，空洞率为 $1, 2, 4, \ldots, 512$（共 10 层），16 kHz 采样，感受野是多少？

答案

$R = 1 + (1 + 2 + \cdots + 512) = 1 + 1023 = 1024$ 个采样点 $\approx$ **64 ms**。堆叠 3 组后
$R = 1 + 3 \times 1023 = 3070 \approx$ **192 ms**，足以覆盖一个音节的局部结构。

课堂练习：WaveNet 能“看”多远？

感受野公式

卷积核大小为 k、空洞率依次为 $d_1, d_2, \ldots$ 的因果卷积堆叠，感受野为

$$R = 1 + \sum_{l} (k-1)\, d_l$$

问题：$k = 2$，空洞率为 $1, 2, 4, \ldots, 512$（共 10 层），16 kHz 采样，感受野是多少？

答案

$R = 1 + (1 + 2 + \cdots + 512) = 1 + 1023 = 1024$ 个采样点 $\approx$ **64 ms**。堆叠 3 组后
$R = 1 + 3 \times 1023 = 3070 \approx$ **192 ms**，足以覆盖一个音节的局部结构。

致命缺点：慢

生成 1 秒 16 kHz 音频需要 **16000** 次串行的网络前向计算，早期实现远远慢于实时。质量革命 + 速度灾难 $\Rightarrow$ 下一步：让声码器并行起来。

让声码器并行起来

更轻的自回归

WaveRNN（2018）

单层 RNN + 稀疏化，一次生成多个采样点，可在 CPU 上实时运行

流模型

WaveGlow（2019）

训练：波形 → 高斯噪声

推理：噪声 $\xrightarrow{\text{逆变换}}$ 波形，一次完成

模型大、显存占用高

GAN

MelGAN、**HiFi-GAN**（2019–2020）

卷积网络一次输出整段波形，判别器负责“挑刺”

又快又好，成为主流

扩散模型

DiffWave、**WaveGrad**（2020）

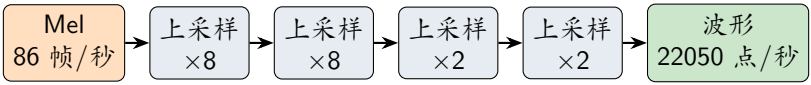
从噪声出发逐步去噪

质量高、训练稳定，但需要多步迭代

共同点

都不再“一个点一个点地”生成，而是一次（或少数几步）生成整段波形，充分利用 GPU 并行计算。

HiFi-GAN（2020）：当前最常用的声码器之一



$8 \times 8 \times 2 \times 2 = 256$：每帧 Mel 恰好对应 256 个采样点（即 STFT 的帧移）

生成器

转置卷积逐级上采样，每级之后用多感受野融合模块（MRF）同时捕捉短、中、长范围的波形模式。

损失函数

$\mathcal{L}_G = \mathcal{L}_{\text{adv}} + 2\,\mathcal{L}_{\text{fm}} + 45\,\mathcal{L}_{\text{mel}}$

对抗损失 + 判别器特征匹配 + Mel 频谱 L1 重建

判别器：从多个角度“挑刺”

- 多周期判别器：按周期 $p \in \{2, 3, 5, 7, 11\}$ 把波形折成二维，检查周期性结构（浊音的基频是周期性的）
- 多尺度判别器：在不同下采样率上检查整体与细节

效果

音质接近真人录音；GPU 上比实时快上百倍，小模型版本在 CPU 上也能实时运行。

声码器演进总结

阶段	代表	音质	速度	关键思想
信号处理	Griffin-Lim, WORLD	中低	快	相位估计 / 源-滤波器
自回归	WaveNet, WaveRNN	高	慢	逐点建模 $p(s_t \mid s_{<t}, c)$
流模型	WaveGlow	高	快（GPU）	可逆变换，一次生成
GAN	MelGAN, HiFi-GAN	高	很快	对抗训练 + 多判别器
扩散	DiffWave, WaveGrad	高	中（多步）	逐步去噪

今天的实践

HiFi-GAN 及其后继（BigVGAN、Vocos 等）是最常见的选择。大模型 TTS 里的“音频编解码器解码器”本质上也是一个 GAN 声码器。

为什么说这是工程上的决定性突破？

并行神经声码器让“接近真人的音质”第一次能在手机和普通服务器上实时运行，TTS 因此得以大规模应用。

目录

1. 任务定义与核心难题
2. 文本前端：读什么
3. 传统语音合成：拼接与统计参数
4. 声学模型：从文字到频谱
5. 声码器：从频谱到波形
6. 端到端合成：VITS
7. 如何评价一个 TTS 系统
8. 展望：大模型时代的 TTS
9. 总结

为什么还要端到端？

两阶段系统的问题

- 训练与推理不匹配：声码器训练时看到的是真实 Mel，推理时却拿到声学模型预测的 Mel（更模糊）
- 通常要用预测的 Mel 再微调声码器
- 误差累积：两个模型分别训练，各自的误差叠加
- Mel 频谱这种人为设定的中间表示不一定是最优的

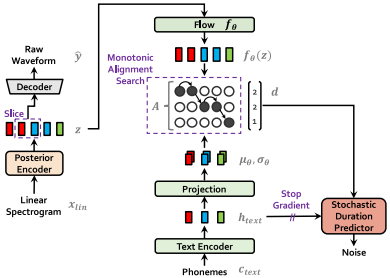
VITS（2021）的回答

- 一个模型：文本 $\rightarrow$ 波形
- 中间表示不再是 Mel，而是网络自己学到的隐变量 z
- 联合训练，没有不匹配问题
- 模型内部自动学对齐，不需要外部对齐器

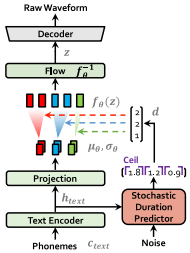
一句话概括

VITS = VAE（学隐变量）+ Flow（增强先验）+ GAN（提升音质）+ MAS（自动对齐）+ 随机时长（节奏多样）

VITS：训练与推理



训练：同时看文本和真实语音



推理：只有文本

图源：J. Kim et al., VITS, ICML 2021

核心思路

训练时：后验编码器从真实语音得到 z（“看答案”），先验编码器学习从文本预测 z 的分布，二者用 KL 散度拉近。

推理时：只用文本侧，预测时长 $\rightarrow$ 采样 z $\rightarrow$ 解码器直接输出波形。

VITS 的各个部件分别解决什么问题？

部件	做什么	对应难题
后验编码器	训练时从真实语音（线性谱）提取隐变量 z	提供学习目标
文本编码器 + Flow	从音素预测 z 的分布；Flow 让分布更灵活	③怎么读
单调对齐搜索（MAS）	动态规划找出文本与 z 的最佳单调对齐	②读多长
随机时长预测器	学习时长的分布，每次采样节奏略有不同	②③节奏多样
解码器（HiFi-GAN 生成器）	把 z 直接变成波形	④怎么发声
判别器	对抗训练，提升波形真实感	④音质

对比 FastSpeech 2

FastSpeech 2 的时长来自外部强制对齐器、是确定的；VITS 的时长在模型内部自动学到，而且是随机的，同一句话每次合成的节奏都可能略有不同，更像真人。

单调对齐搜索（MAS）：又见动态规划

目标

在所有单调、不跳跃的对齐 A 中，找到让隐变量似然最大的那个：

$$A^* = \arg\max_{A} \ \log p(z \mid c_{\text{text}}, A)$$

递推（i：音素，j：帧）

$$Q_{i,j} = \max\left(Q_{i-1,j-1},\, Q_{i,j-1}\right) + \log p(z_j \mid c_i)$$

第 j 帧要么继续读第 i 个音素，要么从第 $i-1$ 个音素前进到第 i 个。

得到时长

回溯得到对齐矩阵后，每个音素的时长就是它占了几帧：

$$d_i = \sum_j A_{i,j}$$

这些时长再用来训练时长预测器。

【回顾】这与第 04 讲 HMM 的维特比算法几乎一样：都是在“只能停留或前进”的约束下，用动态规划找最优路径。

VITS 的效果

系统（**LJSpeech**）	**MOS**	类型
真人录音	4.46	–
VITS	**4.43**	端到端
Glow-TTS + HiFi-GAN（声码器微调后）	4.14（4.32）	两阶段
Tacotron 2 + HiFi-GAN（声码器微调后）	3.77（4.25）	两阶段

从表中能读出什么？

- 括号里的提升说明两阶段系统确实存在训练-推理不匹配
- 端到端联合训练后，几乎追平真人录音

影响

VITS 结构紧凑、推理快，开源后被社区广泛使用，许多中文 TTS、歌声转换和“AI 角色配音”项目都基于它的变体。

课堂试听：用耳朵感受十年的进步

论文官方 Demo 页面

- Tacotron 2：
  `https://google.github.io/tacotron/publications/tacotron2/`
- FastSpeech 2：`https://speechresearch.github.io/fastspeech2/`
- HiFi-GAN：`https://jik876.github.io/hifi-gan-demo/`
- VITS：`https://jaywalnut310.github.io/vits-demo/index.html`

边听边想

1. 哪些样本能听出“机器味”？问题出在韵律（难题③）还是音质（难题④）？
2. FastSpeech 2 调整音高和语速后，哪些地方变得不自然了？

目录

1. 任务定义与核心难题
2. 文本前端：读什么
3. 传统语音合成：拼接与统计参数
4. 声学模型：从文字到频谱
5. 声码器：从频谱到波形
6. 端到端合成：VITS
7. 如何评价一个 TTS 系统
8. 展望：大模型时代的 TTS
9. 总结

主观评价：请人来听

MOS（平均意见得分）

5	优：与真人无异
4	良：有轻微瑕疵
3	中：瑕疵明显但可接受
2	差：瑕疵令人不适
1	劣：非常难听

报告平均分和 **95%** 置信区间，如 4.43 ± 0.06

其他常用形式

- **CMOS**：两个系统听同一句，打 $-3 \sim +3$ 分比较
- **AB** 偏好测试：A 和 B 哪个更好？
- **SMOS**：与目标说话人的音色有多像？

注意事项

听者数量要足够（通常 20 人以上）、评分标准要统一、播放条件要可控；不同论文的 **MOS** 不能直接比较。

客观评价：让机器来测

维度	指标	怎么算	联系
可懂度	CER / WER	用 ASR 转写合成语音，与输入文本对比	第 04 讲
音色相似度	SIM	说话人嵌入的余弦相似度（合成 vs 参考）	第 06 讲
自然度	UTMOS 等	训练一个神经网络预测 MOS	–
信号误差	MCD、F0 RMSE	与同一句真人录音逐帧比较（需对齐）	第 03 讲
效率	RTF、首包延迟	合成耗时 / 音频时长；第一段音频的等待时间	–

为什么需要客观指标？

主观听测准确但贵且慢。模型开发时每天要比较很多版本，需要能自动、快速计算的指标；最终结论仍应以听测为准。

练习与陷阱：别被数字骗了

练习：实时率 RTF

合成 10 秒音频耗时 0.5 秒：

$$\text{RTF} = \frac{0.5}{10} = 0.05$$

比实时快 20 倍。RTF < 1 才能做实时播放。

指标的陷阱

- ASR 自身也会出错，CER 不完全等于 TTS 的错误
- CER 很低 $\neq$ 自然：平淡的“念经”也能被准确识别
- 测试集、文本规范化方式不同，数字就不可比

常用数据集

数据集	语言	规模	特点
LJSpeech	英语	约 24 小时，13100 句	单一女声，最常用的入门基准
VCTK	英语	约 110 位说话人	多种口音，多说话人基准
LibriTTS	英语	约 585 小时，2456 人	有声书，多说话人
标贝 CSMSC	中文	约 12 小时，10000 句	单一女声，含拼音与韵律标注
AISHELL-3	中文	约 85 小时，218 人	多说话人
Emilia	中英日韩德法	10 万小时级	网络自然语音，用于大模型 TTS

两个明显的趋势

- 从录音棚到真实世界：早期用专业播音员录制的干净数据，现在用海量网络语音
- 规模增长约 **1** 万倍：从几十小时到十万小时，带来了零样本克隆等新能力

目录

1. 任务定义与核心难题
2. 文本前端：读什么
3. 传统语音合成：拼接与统计参数
4. 声学模型：从文字到频谱
5. 声码器：从频谱到波形
6. 端到端合成：VITS
7. 如何评价一个 TTS 系统
8. 展望：大模型时代的 TTS
9. 总结

过去三年发生了什么？

数据

几十小时干净录音
⇓
数万到数十万小时真实语音

能力

只能合成训练过的声音
⇓
3 秒参考音频即可克隆音色；跨语言；用自然语言描述控制风格

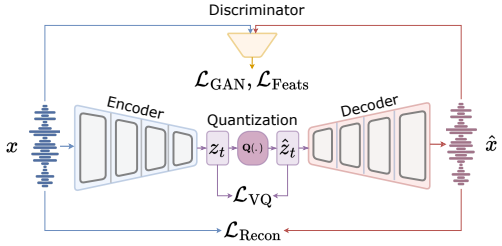
方法

回归连续的 Mel 频谱
⇓
把语音变成离散 token，用语言模型生成

关键问题

语言模型擅长生成离散的 token 序列（文字）。如何把语音也变成 **token**？

神经音频编解码器：把语音变成 token



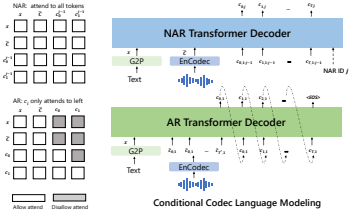
结构

- 编码器：波形 → 每帧一个向量
- 量化器：向量 → 码本中最近的编号（token）
- 解码器：token → 波形（本质上是一个 GAN 声码器）

以 EnCodec（24 kHz）为例

- 每秒 75 帧，每帧 8 个 token
- 残差向量量化（**RVQ**）：第 1 个码本粗略近似，后面的码本逐级修正剩余误差
- 码率约 6 kbps，约为原始 PCM 的 $1/64$

VALL-E（2023）：把 TTS 当作语言建模



图源：C. Wang et al., VALL-E, 2023

做法

- 输入：音素 + 3 秒参考语音的 token
- 自回归模型生成第 1 层 token，非自回归模型补全其余 7 层
- 编解码器的解码器把 token 还原为波形

与本讲的联系

- 像 Tacotron 2 一样自回归，靠结束符决定长度
- 预测 token 的分布再采样，天然处理一对多
- 6 万小时数据训练，参考音频直接作“提示”

今天的主流路线（下一讲详细展开）

自回归语言模型

文本 → 语音 token

VALL-E、Spark-TTS、IndexTTS……

可流式输出，可直接接入大语言模型

非自回归流匹配

文本 + 噪声 → Mel 频谱

E2 TTS、F5-TTS……

结构极简，甚至不需要时长模型和 G2P

混合式

语言模型生成语义 token → 流匹配生成 Mel → 声码器

CosyVoice 系列……

依然是“三段式”的影子

理解新模型的方法

无论名字多新，都可以问：它怎样解决读什么、读多长、怎么读、怎么发声？哪些部件沿用了 Mel 频谱、HiFi-GAN、时长建模这些“老朋友”？

技术之外：安全与伦理

风险

- 几秒钟录音就能克隆一个人的声音
- 电话诈骗、冒充身份、伪造录音
- 未经授权使用他人声音，侵犯声音权益
- 攻击声纹识别系统（联系第 06 讲）

应对

- 知情同意：采集和使用声音需获得本人授权
- 标识：《人工智能生成合成内容标识办法》自 2025 年 9 月起施行，要求对合成语音等内容添加标识
- 技术手段：音频水印、合成语音检测

与课程项目的联系

本课程的 Project 选题中包含“合成语音检测”和“方言 TTS”，欢迎感兴趣的同学结合本讲内容选择。

目录

1. 任务定义与核心难题
2. 文本前端：读什么
3. 传统语音合成：拼接与统计参数
4. 声学模型：从文字到频谱
5. 声码器：从频谱到波形
6. 端到端合成：VITS
7. 如何评价一个 TTS 系统
8. 展望：大模型时代的 TTS
9. 总结

四个核心难题，三代解决方案

难题	传统方法	神经网络 **TTS**	大模型 **TTS**
①读什么	规则 + 词典	神经 G2P、多音字消歧	直接输入字符，从数据中学
②读多长	HMM 状态时长	注意力 / 时长预测 / MAS	自回归结束符 / 指定总时长
③怎么读	统计平均（过平滑）	补充音高能量 + 生成模型	海量数据 + 采样 + 参考音频提示
④怎么发声	拼接录音 / 源-滤波器	WaveNet → HiFi-GAN	编解码器解码器、流匹配 + 声码器

方法在变，问题不变。抓住问题，就能读懂任何一个新模型。

本讲小结

1. TTS 是 ASR 的逆问题，但本质上是一对多的生成问题
2. 经典流水线：文本前端（读什么）→ 声学模型（读多长、怎么读）→ 声码器（怎么发声）
3. 传统方法：拼接合成音质好但不灵活；统计参数合成灵活但过平滑
4. 声学模型：Tacotron 2 用注意力边生成边对齐；FastSpeech 2 用显式时长实现并行、稳定、可控
5. 声码器：WaveNet 带来质量突破，HiFi-GAN 等并行声码器做到又好又快
6. VITS 把各部分合成一个端到端模型，用 MAS 自动学习对齐
7. 评价要同时看可懂度（CER）、自然度（MOS）、相似度（SIM）和效率（RTF）

带走的一句话

语音合成不是“把字一个个读出来”，而是在无数种可能的读法中，生成一种读得对、听着自然、符合要求的声音。

课后思考题

1. 为什么 ASR 用最大似然训练、输出概率最大的文本没有问题，而 TTS 用均方误差回归 Mel 频谱会导致“过平滑”？请用“多对一”和“一对多”解释。
2. FastSpeech 通过显式时长消除了跳词和重复。它失去了什么？VITS 是如何弥补的？
3. HiFi-GAN 的上采样倍数为什么必须等于 STFT 的帧移？如果采样率改为 24 kHz、帧移改为 300，可以怎样设计上采样倍数？
4. 设计一个评测方案，比较两个中文 TTS 系统在“有声书朗读”场景下的表现。你会选哪些指标？测试文本应包含哪些类型的句子？

开源工具与推荐阅读

开源工具

- **ESPnet**、**PaddleSpeech**、**Amphion**：包含 Tacotron 2、FastSpeech 2、VITS、HiFi-GAN 等完整实现
- **Montreal Forced Aligner**：强制对齐，得到音素时长
- **WeTextProcessing**、**pypinyin**、**g2pW**：中文文本规范化与字音转换
- **CosyVoice**、**F5-TTS**：可直接体验的大模型 TTS

推荐阅读

- X. Tan et al., *A Survey on Neural Speech Synthesis*, 2021（本讲多张图的来源）
- X. Tan, *Neural Text-to-Speech Synthesis*, Springer, 2023
- P. Taylor, *Text-to-Speech Synthesis*, Cambridge University Press, 2009（传统方法的经典教材）

参考文献 I

[1] A. J. Hunt and A. W. Black, "Unit Selection in a Concatenative Speech Synthesis System Using a Large Speech Database," *ICASSP*, 1996.

[2] H. Zen, K. Tokuda, and A. W. Black, "Statistical Parametric Speech Synthesis," *Speech Communication*, 2009.

[3] D. Griffin and J. Lim, "Signal Estimation from Modified Short-Time Fourier Transform," *IEEE Trans. ASSP*, 1984.

[4] M. Morise, F. Yokomori, and K. Ozawa, "WORLD: A Vocoder-Based High-Quality Speech Synthesis System for Real-Time Applications," *IEICE Trans. Inf. & Syst.*, 2016.

[5] A. van den Oord et al., "WaveNet: A Generative Model for Raw Audio," *arXiv:1609.03499*, 2016.

[6] Y. Wang et al., "Tacotron: Towards End-to-End Speech Synthesis," *Interspeech*, 2017.

[7] J. Shen et al., "Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions," *ICASSP*, 2018.

[8] N. Kalchbrenner et al., "Efficient Neural Audio Synthesis," *ICML*, 2018.

[9] Y. Jia et al., "Transfer Learning from Speaker Verification to Multispeaker Text-To-Speech Synthesis," *NeurIPS*, 2018.

[10] Y. Wang et al., "Style Tokens: Unsupervised Style Modeling, Control and Transfer in End-to-End Speech Synthesis," *ICML*, 2018.

[11] R. Prenger, R. Valle, and B. Catanzaro, "WaveGlow: A Flow-based Generative Network for Speech Synthesis," *ICASSP*, 2019.

[12] Y. Ren et al., "FastSpeech: Fast, Robust and Controllable Text to Speech," *NeurIPS*, 2019.

[13] K. Kumar et al., "MelGAN: Generative Adversarial Networks for Conditional Waveform Synthesis," *NeurIPS*, 2019.

[14] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis," *NeurIPS*, 2020.

[15] J. Kim, S. Kim, J. Kong, and S. Yoon, "Glow-TTS: A Generative Flow for Text-to-Speech via Monotonic Alignment Search," *NeurIPS*, 2020.

[16] Y. Ren et al., "FastSpeech 2: Fast and High-Quality End-to-End Text to Speech," *ICLR*, 2021.

[17] Z. Kong et al., "DiffWave: A Versatile Diffusion Model for Audio Synthesis," *ICLR*, 2021.

参考文献 II

[18] J. Kim, J. Kong, and J. Son, "Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech," *ICML*, 2021.

[19] X. Tan, T. Qin, F. Soong, and T.-Y. Liu, "A Survey on Neural Speech Synthesis," *arXiv:2106.15561*, 2021.

[20] M. McAuliffe et al., "Montreal Forced Aligner: Trainable Text-Speech Alignment Using Kaldi," *Interspeech*, 2017.

[21] Y.-C. Chen et al., "g2pW: A Conditional Weighted Softmax BERT for Polyphone Disambiguation in Mandarin," *Interspeech*, 2022.

[22] T. Saeki et al., "UTMOS: UTokyo-SaruLab System for VoiceMOS Challenge 2022," *Interspeech*, 2022.

[23] A. Défossez et al., "High Fidelity Neural Audio Compression," *TMLR*, 2023.

[24] C. Wang et al., "Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers," *arXiv:2301.02111*, 2023.

[25] Z. Du et al., "CosyVoice: A Scalable Multilingual Zero-shot Text-to-speech Synthesizer based on Supervised Semantic Tokens," *arXiv:2407.05407*, 2024.

[26] Y. Chen et al., "F5-TTS: A Fairytaler that Fakes Fluent and Faithful Speech with Flow Matching," *arXiv:2410.06885*, 2024.

[27] H. He et al., "Emilia: An Extensive, Multilingual, and Diverse Speech Dataset for Large-Scale Speech Generation," *IEEE SLT*, 2024.